# REPORTE DE PRACTICA NO.1

Autómatas y Compiladores 6° 3

Dr. Eduardo Cornejo-Vel´azquez

Actividad 1.2 Práctica 1

Vite Juarez Alberick

Licenciatura en Ciencias Computacionales

# *Introducción*

Un analizador léxico forma parte fundamental del proceso de compilación de un programa, siendo la primera fase encargada de leer el código fuente y dividirlo en unidades mínimas llamadas tokens. Estos tokens representan elementos significativos del lenguaje de programación, como palabras, números, identificadores y símbolos, los cuales serán utilizados por las siguientes fases del compilador para su análisis sintáctico y semántico.

Un token puede definirse como la unidad básica de información dentro de un programa, a la cual se le asigna un nombre y un valor dependiendo de su contexto. Por ejemplo, una cadena de caracteres compuesta únicamente por letras puede ser clasificada como una palabra, mientras que una secuencia de dígitos se reconoce como un número.

De manera similar, un identificador es un token que cumple con reglas específicas del lenguaje de programación.

En la práctica se simulará el funcionamiento de un analizador léxico con el objetivo de comprender cómo un programa es capaz de leer una cadena de caracteres y clasificarla correctamente en distintos tipos de tokens. Para ello, se desarrolla un programa que permite identificar y diferenciar entre palabras que inician con letra mayúscula, palabras que inician con letra minúscula, números, identificadores y errores léxicos. El programa se implementa utilizando el lenguaje de programación C++, haciendo uso de librerías estándar que facilitan el análisis de caracteres.

# *Marco Teórico*

**Análisis Léxico**

El análisis léxico es la primera etapa dentro de un sistema de compilación. Su función principal consiste en leer el código fuente carácter por carácter y agruparlos en tokens válidos según las reglas definidas por el lenguaje de programación. Esta etapa simplifica el trabajo de las fases posteriores del compilador, ya que transforma una secuencia de caracteres en una secuencia estructurada de tokens.

**Token**

Un token es la unidad mínima con significado dentro de un lenguaje de programación. Está compuesto por un tipo y un valor, donde el tipo indica la categoría a la que pertenece (número, palabra, identificador, etc.) y el valor corresponde a la secuencia de caracteres leída del código fuente. Los tokens permiten que el compilador interprete correctamente las instrucciones escritas por el programador.

**Clasificación de Tokens**

Durante el análisis léxico, los tokens pueden clasificarse en distintas categorías. En esta práctica se consideran los siguientes

**tipos:**

- Número: secuencia compuesta únicamente por dígitos.
- Palabra: secuencia formada solo por letras.
- Identificador: secuencia que inicia con una letra o guion bajo y puede contener letras, números o guiones bajos.
- Error léxico: secuencia que no cumple con las reglas establecidas para los tokens válidos.

**Recorrido Secuencial de Cadenas**

Para la identificación de los tokens, el analizador léxico realiza un recorrido secuencial de la cadena de entrada, evaluando cada carácter de manera individual. Este recorrido permite detectar la presencia de letras, números o símbolos, así como identificar el carácter inicial del token, información fundamental para su correcta clasificación.

**Reglas y Prioridad de Evaluación**

Un mismo token puede cumplir con más de una regla de clasificación, por lo que es necesario establecer un orden de prioridad al momento de evaluar las condiciones. La correcta jerarquía de reglas evita clasificaciones erróneas y asegura que cada token sea identificado de acuerdo con su significado dentro del lenguaje.

# *Herramientas empleadas*

El programa fue desarrollado y probado en un entorno de desarrollo integrado (IDE) para C++, haciendo uso de un compilador compatible con el estándar del lenguaje, lo cual permitió verificar el correcto funcionamiento del analizador léxico implementado.

Lenguaje de Programación: C++

Librerías utilizadas:

- <iostream> → entrada y salida
- <string> → manejo de cadenas
- <cctype> → análisis de caracteres (letras, números, etc.)

# *Desarrollo*

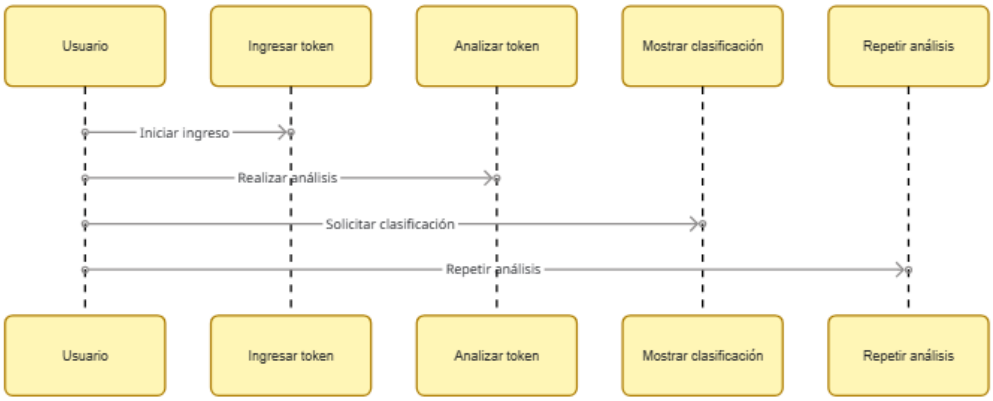
Requisitos del analizador léxico:

| Numero | Función |
|---|---|
| 1 | Leer el Token |
| 2 | Identificar palabra con Mayúsculas |
| 3 | Identificar palabra con Minúscula |
| 4 | Identificar Numero |
| 5 | Identificar Identificadores/variables |

Requisitos opcionales (Personales)

| Numero | Función |
|---|---|
| 1 | Identificar errores |
| 2 | Poder usar continuamente el programa |
| 3 | Mostrar en pantalla el proceso |

**Diagrama de Caso:**




Actor: <u>Usuario</u>
Casos de uso:

- Ingresar token
- Analizar token
- Mostrar clasificación
- Repetir análisis

Durante el desarrollo del analizador léxico, se definió una tabla de clasificación de tokens con el objetivo de establecer reglas claras para identificar el tipo de cada token ingresado por el usuario. Esta tabla permite determinar la categoría del token en función del tipo de caracteres que lo componen y del carácter inicial, lo cual evita ambigüedades en la clasificación.

El analizador realiza un recorrido secuencial del token y, mediante banderas lógicas, identifica si contiene letras, números o símbolos, así como el tipo de carácter con el que inicia. Una vez finalizado el recorrido, se aplica la tabla de clasificación respetando un orden de prioridad, asegurando que cada token sea clasificado correctamente.

| Tipo de Token | Carácter Inicial | Contenido del Token | Restricciones | Ejemplo |
|---|---|---|---|---|
| **Número válido** | Dígito | Solo dígitos | No debe contener letras ni símbolos | 123 |
| **Número mal declarado** | Dígito | Dígitos y letras o símbolos | No cumple la regla de número válido | 12a, 9var |
| **Palabra** | Letra | Solo letras | No debe contener números ni símbolos | hola, Hola |
| **Identificador** | Letra o _ | Letras, números y _ | No debe iniciar con dígito ni contener símbolos | var3, _aux1 |
| **Error léxico** | Cualquiera | Caracteres no válidos | Contiene símbolos no permitidos | hola@, 3#4 |

## Conclusion

El analizador léxico desarrollado cumplió correctamente con los objetivos planteados, ya que fue capaz de identificar y clasificar de forma adecuada los distintos tipos de tokens ingresados por el usuario. Mediante el análisis carácter por carácter, el programa logró diferenciar entre palabras, números, identificadores y errores léxicos, simulando el funcionamiento básico de un analizador léxico real.

Además, se incorporaron mejoras funcionales como la estructura do-while, la cual permite realizar múltiples análisis de tokens en una sola ejecución, haciendo el programa más interactivo y práctico.

## Referencias


Aho, A. V., Ravi, S. A. V., Ullman, J. D. (1998). Compiladores: Principios, técnicas y herramientas. Addison Wesley Longman. México.

Giró, J., Vázquez, J., Meloni, B., Constable, L. (2015). Lenguajes formales y teoría de automátas. Editorial Alfaomega. Argentina.

OpenAI.(2024).*ChatGPT (GPT-4)* [Modelo de lenguaje de gran escala].

Universidad Autónoma del Estado de Hidalgo. (s. f.). *MANUAL DE PRÁCTICAS DE AUTÓMATAS Y COMPILADORES* [Archivo PDF]. https://lms3.uaeh.edu.mx/lms/EJ31026/Licenciatura/pluginfile.php/186053/mod_resource/content/1/Manual.pdf